Hello Payment Solutions

Протокол взаимодействия

Версия 1.4.3

2025

Оглавление

История версий ... 4

Общее описание протокола обмена ... 6

Базовые понятия ... 6

Настройка доступа пользователя в случае его прямого доступа к сервису, либо учетной записи сервиса партнера ... 7

Настройка передачи одноразовых паролей с использованием бота Telegram ... 7

Алгоритм формирования хеша пароля HASHPASS ... 7

Алгоритм формирования API-SIGN ... 8

Процесс авторизации при подключении сервера ... 9

Процесс авторизации при подключении пользователя ... 9

Методы ... 11

Авторизация пользователя – login ... 11

Получение информации о пользователе и доступных ему точках – getUserInfo ... 11

Изменение пароля пользователя – setPass ... 12

Получение баланса – getBalance ... 12

Запрос на генерацию QR кода для оплаты картой – getQRcode ... 12

Запрос статуса оплаты картой по QR коду – getQRstatus ... 13

Запрос на проведение оплаты картой – startCardPay ... 14

Запрос статуса оплаты картой – statusCardPay ... 15

Получение истории операций – getTrx ... 15

Запрос на вывод средств – addPayout ... 17

Отмена запроса на вывод средств – cancelPayout ... 17

Подтверждение вывода средств – commitPayout ... 17

Получение истории операций вывода – getPayout ... 18

Получение справочника способов вывода – getPayoutSources ... 19

Получение дерева групп сервисов – getGroups ... 20

Получение списка сервисов – getServices ... 20

Получение списка сервисов – getServicesV2 ... 21

Получение списка сервисов и групп в одном запросе – getGroups_with_Services ... 22

Запрос на оплату сервиса картой по QR коду СБП – startServiceQRPay ... 23

Запрос статуса оплаты сервиса картой по QR коду СБП – statusServiceQRPay ... 24

Запрос на проведение оплаты сервиса картой – startServiceCardPay ... 24

Запрос статуса оплаты сервиса картой – statusServiceCardPay ... 25

Уведомление об изменении статуса платежей (callback) ... 26

Запрос на пополнение баланса мерчанта (payin Вьетнам, Корея, Япония) – startHVPayin ... 28

Запрос статуса пополнения баланса мерчанта (payin Вьетнам, Корея, Япония) – statusHVPayin ... 29

Получение списка банков для payout Вьетнам, Корея, Япония – getHVBanks ... 30

Запрос на оплату сервисов из предоплаченного баланса мерчанта (payout Вьетнам, Корея, Япония) – startHVPayout 30
Запрос статуса оплаты сервисов из предоплаченного баланса мерчанта (payout Вьетнам, Корея, Япония) – statusHVPayout 31
Запрос на оплату сервисов из предоплаченного баланса – startServicePay 32
Запрос статуса оплаты сервиса из предоплаченного баланса – statusServicePay 32
Запрос на оплату картой HUMO/UZCARD (payin Узбекистан) – startPWPayin 33
Запрос статуса оплаты картой HUMO/UZCARD (payin Узбекистан) – statusPWPayin 35
Запрос на пополнение карт HUMO/UZCARD (payout Узбекистан) – startPWPayout 35
Запрос статуса пополнения карт HUMO/UZCARD (payout Узбекистан) – statusPWPayout 36
Запрос на оплату картой Visa/MasterCard с PCI DSS – startVisaMastercardPayinPCIDSS 37
Передача данных карты при оплаты картой Visa/Mastercard с PCI DSS – sendCardInfo 38
Запрос статуса оплаты картой Visa/Mastercard – statusVisaMastercardPayin 38
Запрос на пополнение карт Visa/Mastercard – startVisaMastercardPayout 39
Запрос статуса пополнения карт Visa/Mastercard – statusVisaMastercardPayout 40
Запрос доступного количества Giftcard для покупки – getServiceInfo 41
Резерв требуемого количества Giftcard – reserveGiftcard 41
Подтверждение оплаты резерва Giftcard – confirmGiftcardPaid 42
Запрос статуса Giftcard, а также их реквизитов – statusGiftcard 42

История версий

Версия 1.1.2 от 25/09/2023

- Добавлен callback на события обработки платежей
- Добавлено описание работы с ботом Telegram для настройки подключения клиентов
- Добавлено описание сервиса расчета hashpass для нового пользователя (в случае использования описанного в данном API алгоритма формирования hashpass)
- Настройка шлюза Telegram для передачи одноразовых паролей в Telegram пользователей
- Уточнено описание процесса авторизации

Версия 1.1.3 от 17/10/2023

- Добавлена оплата картами (эквайринг) – *CardPay

Версия 1.2.1 от 23/10/2023

- Добавлена оплата сервисов картами (эквайринг) – *ServiceCardPay
- Переименованы процедуры оплаты сервисов через QR СБП – *ServiceQRPay (старые названия оставлены для совместимости)

Версия 1.3.1 от 22/11/2023

- Добавлено пополнение баланса мерчанта оплатой со счетов (payin Вьетнам, Корея, Япония) – *HVPayin
- Добавлено получение списка банков для payout Вьетнам, Корея, Япония – getHVBanks
- Добавлена оплата сервисов из предоплаченного баланса мерчанта (payout Вьетнам, Корея, Япония) – *HVPayout
- Оплата сервисов из предоплаченного баланса *ServicePay

Версия 1.3.2 от 14/02/2024

- Добавлено поле source для payin Вьетнам (пополнение nganluong)
- Добавлена ЭЦП и статус в callback

Версия 1.3.3 от 23/03/2024

- Добавлен метод оплаты картами HUMO/UZCARD (PayIn Узбекистан) – *PWPayin

Версия 1.3.4 от 29/03/2024

- Добавлен метод пополнения карт HUMO/UZCARD (PayOut Узбекистан) – *PWPayout

Версия 1.3.5 от 04/04/2024

- Добавлены реквизиты в *HVPayin

Версия 1.3.6 от 09/04/2024

- Добавлен тип запроса payin/payout в callback, который будет отправляться GET

Версия 1.3.7 от 24/07/2024

- Исправлено описание статусов в методе startServiceQRPay и statusServiceQRPay

Версия 1.3.8 от 16/10/2024

- Добавлено описание методов Payin/Payout для Visa, Mastercard

Версия 1.3.9 от 05/11/2024

- Добавлено комментарий по статусу 3 к методам Payin/Payout для Visa, Mastercard

Версия 1.4.1 от 17/12/2024

- Добавлена запросы getServicesV2 и getServiceInfo для покупки ваучеров и т.п. Запрос getServices является устаревшим. Необходимо переходить к использованию новой версии запроса.

Версия 1.4.2 от 24/03/2025

- Добавлен уточняющий комментарий по лимитам в методах HUMO/UZCARD (payin Узбекистан)

Версия 1.4.3 от 07/04/2025

- Добавлена работа с Giftcard.
- Добавлены уточнения по логике работы сессии.

Общее описание протокола обмена

Базовые понятия

По умолчанию запросы POST application/x-www-form-urlencoded, если не указано иное.

Все ответы в формате JSON

Формат datetime “yyyy-mm-dd hh:mm:ss”

Запросы на endpoint типа /hellopay/command

Используется API-KEY авторизация. Каждый HTTP запрос должен содержать заголовки:

API-VER: int32 – версия API
API-KEY: string – логин клиента или случайная строка для подключения серверов
API-SESSION: int32 – int32 – номер сессии который будет передан после логина. При запросе login этот заголовок не передается или можно передать -1. Необходимо работать в созданной сессии, пока не получите ошибку 401, нельзя открывать несколько сессий, если сделаете запрос логина, предыдущая сессия закрывается.
API-POINT: – номер точки (если не известна можно передать -1 или не передавать этот заголовок)
API-TS: int64 – текущее время UTC в миллисекундах. Важно, чтобы на клиенте было точное время. Максимальное отклонение между временем на сервере и клиенте 15 секунд.
API-SIGN: string – HMAC-SHA256 подпись запроса. Алгоритм формирования описан ниже.

При подключении партнера предоставляется строка SALT, которая используется при формировании хеш.

При подключении сервера предоставляются API-POINT, API-KEY и API-PASS.

Для настройки callback на изменения статуса обработки платежей партнер должен сообщить URL конечной точки при подключении.

В случае реализации сервиса партнера в виде бота Telegram, возможна настройка шлюза Telegram для передачи одноразового пароля пользователя при его авторизации. Для включения этого режима необходимо сообщить числовой idTelegram бота, который будет выступать в качестве шлюза.

Необходимо обеспечить безопасное хранение SALT и API-PASS и никому их не сообщать! В явном виде в запросах они не передаются, а используются только для вычисления HMAC-SHA256.

Настройка доступа пользователя в случае его прямого доступа к сервису, либо учетной записи сервиса партнера

В качестве пользователя системы может выступа как конечный пользователь, так и сервис партнера (в этом случае он будет выступать в роли пользователя) в зависимости от реализуемого функционала.

В любом случае для предоставления доступа пользователя к сервису необходимо предоставить логин, **hashpass**

При подключении пользователя с двойной авторизацией, необходимо также предоставить идентификатор пользователя в Telegram.

Настройка передачи одноразовых паролей с использованием бота Telegram

Для использования двухфакторной авторизации пользователю необходимо начать общение с ботом, чтобы бот мог ему присылать сообщения.

При настройке нового пользователя необходимо сообщить администратору имя пользователя в Telegram (например, @petrov_a) или числовой idTelegram пользователя, который можно узнать с помощью бота @IDBot.

Пользователь подключается к боту @HelloPSBot, начинает общение с ним. Если настройка проводилась по имени пользователя, а не по числовому Id, то необходимо в меню выбрать «Authorize by Telegram» для того, чтобы бот определил числовой Id пользователя.

Алгоритм формирования хеша пароля HASHPASS

1. В качестве пароля пользователя используется не сама строка пароля клиента, а base64 hmac-sha256 хеш от строки “username:password” с ключом “salt” или любой другой случайной строки (которая постоянна на стороне партнера) во избежание появления одинаковых хешей паролей разных пользователей с одним паролем и компрометации паролей пользователей. В дальнейшем этот хеш-идентификатор будет обозначаться ***hashpass***.
2. Для расчета ***hashpass*** при подключении пользователя можно воспользоваться сервисом https://my.hellopsp.com/register.html
3. При подключении сервера в качестве ***hashpass*** используется API-PASS.

Алгоритм формирования API-SIGN

Формируется строка for_sign

format('%s:%s:%d:%d:%s', [API-KEY, API-TS, API-POINT, API-SESSION, SALT])

- Для запроса открытия сессии /hellopay/login при авторизации API-SIGN рассчитывается

  base64 hmac-256 от строки for_sign с ключом hashpass

- Для последующих запросов API-SIGN рассчитывается

  base64 hmac-256 от строки for_sign с ключом sessionkey

Процесс авторизации при подключении сервера

1. Параметры авторизации передаются в заголовках API-KEY, API-PASS, API-POINT, API-TS, API-SIGN, API-SESSION (при login передается -1).
2. На сервер отправляется запрос /hellopay/login без параметров.
3. Сервер проверяет авторизацию. В случае ошибки возвращается ошибка 401.
4. Если проверка пройдена успешно, то сервер возвращает код KEY в ответе на запрос для расчета сессионного ключа и номер сессии, который в дальнейшем передается в заголовке API-SESSION, например

   {"result" : "ok","id_session" : "542","key" : "6334104772177609032"}

5. Рассчитывается ***sessionkey***:

   base64 hmac-sha256 хеш строки “API-KEY:API-PASS:SALT” с ключом KEY

Пример на php:

base64_encode(hash_hmac('sha256', $headers['API-KEY'].':'.$headers['API-TS'].':'.$headers['API-POINT'].':'.$headers['API-SESSION'].':'.$this->salt, $pass, true));

Важно понимать, что работа должна проводиться в рамках одной сессии, нельзя:
- Делать login на каждый платеж, именно данный запрос открывает новую сессию
- Открывать новую сессию, те запускать login, если не была получена ошибка 401
- Принудительно менять номера сессий

Алгоритм работы выглядит следующим образом:

1. При инициализации программы создаете переменную API-KEY. Сначала она имеет значение: -1,
2. Если API-KEY = -1, то отправляем запрос login, если получили иной ответ делаете запрос, который необходимо отправить (например, getQRcode).
3. Если в результате отправленного в п.1 запроса получили ошибку http 401, то API-KEY =-1 и идем на п.1

Процесс авторизации при подключении пользователя

4. На сервер отправляется запрос /hellopay/login без параметров. Логин пользователя передается в заголовке API-KEY, а hashpass используется в качестве ключа для hmac-sha256 при формировании API-SIGN.
5. Хранить пароль не нужно.
6. Сервер проверяет авторизацию. В случае ошибки возвращается ошибка 401.
7. Если проверка пройдена успешно, то сервер генерирует два одноразовых кода: длинный “key” и короткий код-пароль “code”. Код-пароль code направляется агенту на почту/в телеграм. А в ответе на запрос сервер возвращает длинный код для расчета сессионного ключа и номер сессии например

   {"result" : "ok","id_session" : "542","key" : "6334104772177609032"}

8. При получении ответа с key интерфейс предлагает пользователю ввести одноразовый пароль “code” из почты/телеграм
9. Рассчитывается ***sessionkey***:

base64 hmac-sha256 хеш строки “username:hashpass:code:salt” с ключом “key”

10. Сервис партнера по необходимости направляет на сервер запрос getUserInfo для получения данных по роли и правам пользователя, а также списка доступных точек.
11. Необходимо предоставить возможность пользователю выбрать, с какой точкой он будет работать. Во всех последующих запросах необходимо передавать заголовок API-POINT: номер_точки

Методы

Авторизация пользователя – login

Конечная точка: **login**

Запрос без дополнительных параметров

Параметры ответа:

- rc:integer – код ошибки, 0 если успешно
- rm: string – “ok”, если успешно
- *id_session:integer – номер сессии*
- *key:string – часть ключа для расчета сессионного ключа sessionkey*

Получение информации о пользователе и доступных ему точках – getUserInfo

Конечная точка: **getUserInfo**

Запрос без дополнительных параметров

Параметры ответа:

- rc:integer – код ошибки, 0 если успешно
- rm: string – “ok”, если успешно
- user_descr:string – имя пользователя
- user_role:string – роль пользователя
- user_rights:string – список номеров прав пользователя (например “2,4,5”)
- data – массив объектов – точек, доступных пользователю. Каждый объект содержит поля:
    - rowid:integer – идентификатор пользователь-точка. Нужен для настроечных запросов
    - id_point: integer – номер точки
    - description:string – название точки
    - id_interface:integer – идентификатор интерфейса
    - name_interface:string – наименование интерфейса
    - email: string – почтовый адрес точки для отправки чеков
    - currency:integer – код валюты агента

Изменение пароля пользователя – setPass

Конечная точка: **setPass**

Параметр pass содержит hashpass зашифрованный AES

Пароль передается в зашифрованном AES-CBC с случайным вектором IV виде:

1. Вычисление ключа AES: aesKey=md5(base64(sessionkey))
2. В качестве IV используется случайная строка 128 бит. Например, IV=md5(GUID).
3. Вычисляется crc передаваемой строки md5(hashpass)
4. Шифруем строку crc+hashpass: encrypted=aes_cbc(crc+hashpass,aesKey,IV)
5. Передаем строку base64(IV+encrypted)

Параметры ответа:

- rc:integer – код ошибки, 0 если успешно
- rm: string – “ok”, если успешно

Получение баланса – getBalance

Конечная точка: **getBalance**

Параметры ответа:

balance:currency – баланс (для оплаты QR сумма доступная для вывода)

balance_on_hold:currency – сумма непроведенных платежей, для QR сумма неоплаченных не просроченных QR кодов

currency:integer – код валюты баланса

Запрос на генерацию QR кода для оплаты картой – getQRcode

Конечная точка: **getQRcode**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в копейках

currency: string – цифровой код валюты платежа по ISO 4217 (рубль – 643)

amount_alg:integer. Если 0, то плательщик комиссий клиент. Если 1, то комиссии платит агент и сумма должна быть точно зачислена клиенту.

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

n:string – строка с информацией о платеже. обычно в ней параметры разделяются символом #. Первый параметр – идентификатор клиента, второй – код услуги по справочнику, остальные – по желанию. Например:

91233432534#12345#Проживание#Номер

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

amount_sbp – сумма для оплаты в копейках, которая направлена в СБП

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку,
- 2 платеж отправлен, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в пс
- От 10 до 99 ошибочные окончательные

sbpstatus:string – статус платежа СБП

qrcId: string – строка, которую необходимо визуализировать в qr код и вывести клиенту для оплаты.

trxId:string – идентификатор успешной транзакции СБП

Запрос статуса оплаты картой по QR коду – getQRstatus

Конечная точка: **getQRstatus**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для getQRcode

Запрос на проведение оплаты картой – startCardPay

Конечная точка: **startCardPay**

Результатом запроса является ссылка на платежную страницу банка url_redirect, на который необходимо перенаправить клиента для оплаты картой. После окончания оплаты клиент автоматически перенаправляется на страницы url_success или url_fail в зависимости от успешности оплаты. В любой момент можно узнать текущий статус оплаты запросом statusCardPay. Адреса страниц url_success и url_fail необходимо сообщить при подключении.

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в копейках

currency: string – цифровой код валюты платежа по ISO 4217 (рубль – 643)

amount_alg:integer. Если 0, то плательщик комиссий клиент. Если 1, то комиссии платит агент и сумма должна быть точно зачислена клиенту.

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

n:string – строка с информацией о платеже. обычно в ней параметры разделяются символом #. Первый параметр – идентификатор клиента, второй – код услуги по справочнику, остальные – по желанию. Например:

91233432534#12345#Проживание#Номер

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

amount_account – сумма для оплаты в копейках, которая направлена в банк

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку,
- 2 платеж отправлен, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

url_redirect:string – ссылка на платежную страницу банка, на которую необходимо перенаправить клиента для оплаты картой

bank_trxid:string – идентификатор транзакции в банке (появляется после перехода на платежную страницу)

bank_status:string – описание ошибок платежа в банке

amount_payed:integer – сумма в копейках, оплаченная клиентом

srctype:string – тип оплаты. Для оплаты картой – CARD

paymentsystem:string – платежная система, например VISA

maskedpan:string – маскированный номер карты клиента

authcode:string – код авторизации

rrn:string – уникальный идентификатор банковской транзакции.

Запрос статуса оплаты картой – statusCardPay

Конечная точка: **statusCardPay**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startCardPay

Получение истории операций – getTrx

Конечная точка: **getTrx**

Для точки выдаются только операции, проведенные на этой точке. Для агента выдаются операции по всем точкам, относящихся к этому агенту.

Дополнительные параметры запроса:

point: number – идентификатор точки

page: number – номер страницы. Опциональный параметр

rows: number – количество операций выдаваемых за один раз (по умолчанию 10. Опциональный параметр

ts_start: DateTime – дата начала периода. Опциональный параметр

ts_finish: DateTime – дата конца периода. Опциональный параметр

id_trx: number – идентификатор транзакции (выборка только одной транзакции). Опциональный параметр

show_all: boolean – если не указано, то не отображаются просроченный QR коды. Если указано show_all=true, то отображаются все транзакции. Опциональный параметр

with_balance: boolean – если указано with_balance=true, то в результатах добавлены информация о балансе. Опциональный параметр

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

rows_total: number – общее количество строк, удовлетворяющее запросу

rows: operation[] – список операций

operation – {

id_trx: integer – идентификатор транзакции

id_trans: string – идентификатор транзакции клиента

point:integer – номер точки, где был платеж

pointname:string – описание точки, где был платеж

sbp_trxid:string – идентификатор транзакции СБП

status: number – код статуса операции (проведена, в обработке и т.д.)

n:string – параметры платежа из запроса amount: currency – сумма

currency_pay: integer – код валюты платежа

amount_ps: currency – сумма, направленная в ПС

currency_ps: integer – код валюты ПС

amount_sbp: currency – сумма, направленная в СБП

ts: string – время платежа

ts_getpay: string – время получения платежа в сервис

ts_confirm: string – время подтверждения проведения платежа

errmsg_pay:string – сообщение от ПС о проведении платежа }

Запрос на вывод средств – addPayout

Конечная точка: **addPayout**

Дополнительные параметры запроса:

note: string – комментарий

amount:integer – сумма для вывода в копейках валюты терминала

pay_src:integer – код способа вывода средств по справочнику

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trx:string – идентификатор транзакции

Отмена запроса на вывод средств – cancelPayout

Конечная точка: **cancelPayout**

Дополнительные параметры запроса:

id_trx:string – идентификатор транзакции

note: string – комментарий

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trx:string – идентификатор транзакции

Подтверждение вывода средств – commitPayout

Конечная точка: **commitPayout**

Дополнительные параметры запроса:

id_trx:string – идентификатор транзакции

note: string – комментарий

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trx:string – идентификатор транзакции

Получение истории операций вывода – getPayout

Конечная точка: **getPayout**

Для точки выдаются только операции, проведенные на этой точке. Для агента выдаются операции по всем точкам, относящихся к этому агенту.

Дополнительные параметры **запроса:**

point: number – идентификатор точки

page: number – номер страницы. Опциональный параметр

rows: number – количество операций выдаваемых за один раз (по умолчанию 10. Опциональный параметр

ts_start: DateTime – дата начала периода. Опциональный параметр

ts_finish: DateTime – дата конца периода. Опциональный параметр

id_trx: number – идентификатор транзакции (выборка только одной транзакции). Опциональный параметр

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

rows_total: number – общее количество строк, удовлетворяющее запросу

rows: operation[] – список операций

operation – {

id_trx: integer – идентификатор транзакции

id_pair_trx: integer – идентификатор парной транзакции клиента

id_paytype: integer – идентификатор типа транзакции

paytype_descr: string – описание типа транзакции

id_paysrc: integer – идентификатор способа вывода средств

src_descr: string – описание способа вывода средств

amount: currency – сумма

note:string – комментарий

}

Получение справочника способов вывода – getPayoutSources

Конечная точка: **getPayoutSources**

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

rows: operation[] – список типов

operation – {

rowid: integer – идентификатор

descr: string – описание способа вывода средств

}

Получение дерева групп сервисов – getGroups

Конечная точка: **getGroups**

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

rows: operation[] – дерево групп

operation – {

rowid: integer – идентификатор группы

parent: integer – идентификатор предка группы в дереве групп

name: string – название группы

parentname: string – название предка группы

}

Получение списка сервисов – getServices

Устарело. Не поддерживается продажа ваучеров и т.п. Необходимо перейти к использованию getServicesV2

Конечная точка: **getServices**

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

rows: operation[] – список сервисов

operation – {

rowid: integer – идентификатор

id_prv_service: integer – идентификатор группы

name: string – название сервиса

divider: integer – делитель единиц измерения

id_operator: integer – идентификатор поставщика услуг (оператора)

group: integer – группа услуг

minamount: integer –минимальная сумма платежа

maxamount: integer – максимальная сумма платежа

opt:integer – вариант набора услуг оператора

fields: field[] – список параметров

field – {

num_field:integer – номер параметра

info:string – Описание параметра на двух языках через ~

Например: *Card number~Номер карты*

logo:string – имя файла логотипа оператора

fieldmask:string – маска для ввода номера или строки. Данные вводятся вместо символа _. Например +374 __ _____. Передавать необходимо только то, что введено.

keyboard_type:integer – тип клавиатуры: 1 – цифровая, 2 – текст

min_text_length: integer – минимальная длина строки, которую необходимо ввести. 0 – необходимо заполнить всю маску

regexp:string – регулярное выражение для проверки корректности введенных данных и формирования маски

}

}

Получение списка сервисов – getServicesV2

Актуальный вариант сервиса. Необходимо его использовать для получения полной информации для продажи ваучеров и т.п.

Конечная точка: **getServicesV2**

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

rows: services[] – список сервисов

service – {

id_service: integer – идентификатор

id_group: integer – идентификатор группы

name: string – название сервиса

country_num: код страны сервиса (нестандартные коды: 1 - Евросоюз, 2 – GLOBAL (все страны),

country_a2: альфа-2 страны сервиса (нестандартные коды: EU - Евросоюз, ZZ – GLOBAL (все страны),

divider: integer – делитель единиц измерения

id_operator: integer – идентификатор поставщика услуг (оператора)

group: integer – группа услуг

minamount: integer –минимальная сумма платежа

maxamount: integer – максимальная сумма платежа

opt:integer – вариант набора услуг оператора

fields: field[] – список параметров

field – {

num_field:integer – номер параметра

info:string – Описание параметра на двух языках через ~

Например: *Card number~Номер карты*

logo:string – имя файла логотипа оператора

fieldmask:string – маска для ввода номера или строки. Данные вводятся вместо символа _. Например +374 __ _____. Передавать необходимо только то, что введено.

keyboard_type:integer – тип клавиатуры: 1 – цифровая, 2 – текст

min_text_length: integer – минимальная длина строки, которую необходимо ввести. 0 – необходимо заполнить всю маску

regexp:string – регулярное выражение для проверки корректности введенных данных и формирования маски

}

}

Получение списка сервисов и групп в одном запросе – getGroups_with_Services

Конечная точка: **getGroups_with_Services**

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

groups: operation[] – дерево групп

services: operation[] – список сервисов }

Запрос на оплату сервиса картой по QR коду СБП – startServiceQRPay

Конечная точка: **startServiceQRPay**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в копейках

currency: string – цифровой код валюты платежа по ISO 4217 (рубль – 643)

amount_alg:integer. Если 0, то плательщик комиссий клиент. Если 1, то комиссии платит агент и сумма должна быть точно зачислена клиенту.

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

id_operator:integer – номер поставщика услуг из справочника, полученного запросом getServices

n:string – строка с информацией о платеже. Список необходимых параметров передается в результате запроса getServices. Например: 91233432534#12345#Проживание#Номер

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 создан запрос на проведение платежа в СБП. Необходимо вывести клиенту QR код для оплаты
- 2 ожидание оплаты QR кода. На оплату дается 72 часа, после оплаты переходит на статус 3, в случае неоплаты в статус 22.
- 3 QR код оплачен

- 100 101 102 промежуточные
- От 10 до 99 ошибочные окончательные

Запрос статуса оплаты сервиса картой по QR коду СБП – statusServiceQRPay

Конечная точка: **statusServiceQRPay**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 создан запрос на проведение платежа в СБП. Необходимо вывести клиенту QR код для оплаты
- 2 ожидание оплаты QR кода. На оплату дается 72 часа, после оплаты переходит на статус 3, в случае неоплаты в статус 22.
- 3 QR код оплачен
- 100 101 102 промежуточные
- От 10 до 99 ошибочные окончательные

amount_sbp: integer – сумма для оплаты в копейках, которая направлена в СБП

sbpstatus: string – статус платежа СБП

qrcId: string – строка, которую необходимо визуализировать в qr код и вывести клиенту для оплаты.

trxId: string – идентификатор успешной транзакции СБП

Запрос на проведение оплаты сервиса картой – startServiceCardPay

Конечная точка: **startServiceCardPay**

Результатом запроса является ссылка на платежную страницу банка url_redirect, на который необходимо перенаправить клиента для оплаты картой. После окончания оплаты

клиент автоматически перенаправляется на страницы url_success или url_fail в зависимости от успешности оплаты. В любой момент можно узнать текущий статус оплаты запросом statusServiceCardPay. Адреса страниц url_success и url_fail необходимо сообщить при подключении.

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в копейках

currency: string – цифровой код валюты платежа по ISO 4217 (рубль – 643)

amount_alg:integer. Если 0, то плательщик комиссий клиент. Если 1, то комиссии платит агент и сумма должна быть точно зачислена клиенту.

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

id_operator:integer – номер поставщика услуг из справочника, полученного запросом getServices

n:string – строка с информацией о платеже. Список необходимых параметров передается в результате запроса getServices. Например: 91233432534#12345#Проживание#Номер

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 создан запрос на проведение платежа в ГПБ. Необходимо переадресовать клиента на страницу банка для ввода реквизитов карты
- 2 Клиент оплатил, платеж направлен поставщику услуг для проведения, ждем статус успешности.
- 3 Оплачено и проведено поставщиком услуг
- 100 101 102 промежуточные
- От 10 до 99 ошибочные окончательные

Запрос статуса оплаты сервиса картой – statusServiceCardPay

Конечная точка: **statusServiceCardPay**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

amount_account – сумма для оплаты в копейках, которая направлена в банк

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку,
- 2 платеж отправлен, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

url_redirect:string – ссылка на платежную страницу банка, на которую необходимо перенаправить клиента для оплаты картой

bank_trxid:string – идентификатор транзакции в банке (появляется после перехода на платежную страницу)

bank_status:string – описание ошибок платежа в банке

amount_payed:integer – сумма в копейках, оплаченная клиентом

srctype:string – тип оплаты. Для оплаты картой – CARD

paymentsystem:string – платежная система, например VISA

maskedpan:string – маскированный номер карты клиента

authcode:string – код авторизации

rrn:string – уникальный идентификатор банковской транзакции.

Уведомление об изменении статуса платежей (callback)

Для оперативного уведомления сервиса партнера об изменении статуса платежей на URL callback, который был предоставлен клиентом, будут направляться запрос GET без авторизации с параметрами в строке запроса:

id_pay – идентификатор транзакции сервиса

id_trans – идентификатор транзакции клиента.

info – различная дополнительная информация

ts – текущее время UTC в миллисекундах

type – тип платежа payin/payout

sign – base64 SHA256 RSA от строки

https://callback_url?id_pay=123&id_trans=345&info=qr_status&ts=1706106701454 до &sign=

На случай разбора возможных недоразумений необходимо сохранять эти запросы с подписью. Необходимо проверять подпись и контролировать, чтобы ts сильно не отличалось от текущего времени.

При получении callback необходимо направить ответ «OK» и после этого сделать запрос статуса для того, чтобы получить полную информацию о платеже.

Делается 10 попыток выполнить запрос GET для отправки Callback с увеличивающимся интервалом 2,4,6,8,… секунд.

Callback будет приходить при значимых изменениях статуса. Например, для сервиса оплаты QR СБП callback будут в моменты:

- сформирован QR – можно его запрашивать и отображать клиенту
- оплачен QR – можно убрать QR с экрана и проводить платеж клиенту
- обработка QR прекращена по какой-то причине (например, прошло 72 часа).

Пример проверки ЭЦП:

openssl.exe enc -base64 -d -in sign.b64 -out sign.bin

openssl.exe dgst -sha256 -verify public.pem -signature sign.bin signed

Сертификат открытого ключа и открытый ключ, которым подписываются колбеки:

-----BEGIN CERTIFICATE-----
MIIEBTCCAu2gAwIBAgIBADANBgkqhkiG9w0BAQQFADBkMQswCQYDVQQGEwJSUzER
MA8GA1UECBMIQmVsZ3JhZGUxETAPBgNVBAcTCEJlbGdyYWRlMRAwDgYDVQQKEwdI
ZWxsb1BTMQswCQYDVQQLEwJJVDEQMA4GA1UEAxMHSGVsbG9QUzAeFw0yNDAxMjQx
MjEzMzhaFw0zNzEwMDIxMjEzMzhaMGQxCzAJBgNVBAYTAlJTMREwDwYDVQQIEwhC
ZWxncmFkZTERMA8GA1UEBxMIQmVsZ3JhZGUxEDAOBgNVBAoTB0hlbGxvUFMxCzAJ
BgNVBAsTAklUMRAwDgYDVQQDEwdIZWxsb1BTMIIBIjANBgkqhkiG9w0BAQEFAAOC
AQ8AMIIBCgKCAQEArC+13nUSEk9zkytXS68if8aH2qIdiwLg4WU/W/YkwwEjVr6B
GgJtcDf3MQcAcDip2KldIauEOcywC8dnYfRQK1fUnMj44stm7jZvcEQCdwg9zJUK
9sVsv1wuzdc+/OuUXwK+fQpUobVvLVvXJlOPDXVQoroveXDyra+eZbRvLMYTOIDK
r3kngQoKlNXdugWMm4wn3hIEUG/ZncbIiVHc1fHbj5rymLa8BZrs0BQVbvTjju0+
BqlYOSoW2CIfhGZ1lig2aETqOJbtuiIjmUOUiy5+tSka9uA9jldCByg8DgGIXV1S
hKE9ul/A9OyCK3OwIpjV34HnfQrkzZMozVKP8wIDAQABo4HBMIG+MB0GA1UdDgQW
BBQ+RvopRgHnBTkLwXSoSPG/vMao2zCBjgYDVR0jBIGGMIGDgBQ+RvopRgHnBTkL
wXSoSPG/vMao26FopGYwZDELMAkGA1UEBhMCUlMxETAPBgNVBAgTCEJlbGdyYWRl
MREwDwYDVQQHEwhCZWxncmFkZTEQMA4GA1UEChMHSGVsbG9QUzELMAkGA1UECxMC
SVQxEDAOBgNVBAMTB0hlbGxvUFOCAQAwDAYDVR0TBAUwAwEB/zANBgkqhkiG9w0B
AQQFAAOCAQEAdFHIbiMPJOTKvP5jwldgcCVS4LfTL1q3cbccBdfStcYHuN5m2h6a
82AvQgkingEDq4crV/tD4Kw2pw+qK6XYXFCmjleo9XQMDTE1c3M2z5gO+MBGtwfo
one7+8sy8ETIkAg7n5QhGdJORgk2j9bS5x5ckaQCW7qS4YXk8IvYowjzmzvlaxLA
yMKUAaeQg2zmCfp629wtMK8q94/L2S/hsxffWKZLaa+gcq8k7mqbAwUf3G1yTq2N
DxKJHrwGBh14MbsnlbJiWb6cSMXBZXeIwbJFJONz3Oz7CFvM+i7rm/OU0KvLtxM6
fJ0XJR83iBv3dzQYmLqtMtKAXEJRueBH/g==
-----END CERTIFICATE-----

-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEArC+13nUSEk9zkytXS68i

f8aH2qIdiwLg4WU/W/YkwwEjVr6BGgJtcDf3MQcAcDip2KldIauEOcywC8dnYfRQ
K1fUnMj44stm7jZvcEQCdwg9zJUK9sVsv1wuzdc+/OuUXwK+fQpUobVvLVvXJlOP
DXVQoroveXDyra+eZbRvLMYTOIDKr3kngQoKlNXdugWMm4wn3hIEUG/ZncbIiVHc
1fHbj5rymLa8BZrs0BQVbvTjju0+BqlYOSoW2ClfhGZ1lig2aETqOJbtuiljmUOU
iy5+tSka9uA9jldCByg8DgGIXV1ShKE9ul/A9OyCK3OwIpjV34HnfQrkzZMozVKP
8wIDAQAB

-----END PUBLIC KEY-----

Запрос на пополнение баланса мерчанта (payin Вьетнам, Корея, Япония) – startHVPayin

Конечная точка: **startHVPayin**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в минимальные единицах платежа страны

currency: string – цифровой код валюты платежа по ISO 4217 (например донг – 704)

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

source:string (необязательное) – по умолчанию «bank», может быть «nganluong»

sender_email: string – email плательщика. Обязательно для Японии, очень желательно для Кореи

sender_first_name: string – имя плательщика. Обязательно для Кореи

sender_last_name: string – фамилия плательщика. Обязательно для Кореи

В случае, если сервис поставщика успеет ответить в течение 30 секунд, то данные для проведения платежа клиентом будут сразу отображены. В противном случае необходимо сделать запрос statusHVPayin через несколько секунд (5-10).

Для оплаты клиенту необходимо сообщить bank_amount, bank_currency_code, name, bank, branch, account, note.

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

amount_payin – сумма заявки для оплаты

amount_payed – итоговая сумма платежей клиента в оплату данного payin

currency_payin – валюта платежа в банк

bank_amount – сумма, на которую сформирован payin в банк

bank_currency_code – валюта платежа в банке

name – имя получателя

bank – наименование банка из справочника

branch – подразделение банка

account – номер счета

note – идентификатор платежа

source – источник платежа (по умолчанию bank)

unixdate – дата платежа в банке в формате Unix

date – дата платежа в текстовом формате

status_hv – статус платежа в платежной системе

comments – комментарии от банка

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку,
- 2 платеж отправлен, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

status_info – дополнительное текстовое описание статуса

Пример ответа:

{"id_trans":"1700401041192", "id_pay":"16662012", "status":"2", "amount_payin":1234, "amount_payed":0, "currency_payin":"vnd", "bank_amount":1234, "bank_currency_code":"vnd", "name":"John Doe", "bank":"VCB", "branch":"VBC-1", "account":"1234567890", "note":"7777 9 4551", "source":"bank", "unixdate":1700401044, "date":"19.11.2023 13:37:24", "status_hv":"new", "comments":"", "status_info":"Wait client pay", "rc":"0", "rm":"process"}

Запрос статуса пополнения баланса мерчанта (payin Вьетнам, Корея, Япония) – statusHVPayin

Конечная точка: **statusHVPayin**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startHVPayin

Получение списка банков для payout Вьетнам, Корея, Япония – getHVBanks

Обновлять список банков имеет смысл раз в сутки.

Конечная точка: **getHVBanks**

Дополнительные параметры запроса:

currency: string – цифровой код валюты платежа по ISO 4217 для определения страны (донг – 704)

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

banks: [bank] – список банков

bank – { id: integer – идентификатор банка

name: string – название банка

branches: список подразделений}

Пример:
[{"id":2441,name":"AGRIBANK",branches:[]},
{"id":2442,"name":"BIDV",branches:[{ "name":"branch_name", "code":"0001" }]}]

Запрос на оплату сервисов из предоплаченного баланса мерчанта (payout Вьетнам, Корея, Япония) – startHVPayout

Конечная точка: **startHVPayout**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в копейках

currency: string – цифровой код валюты платежа по ISO 4217 (донг – 704)

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

name:string – имя получателя

bank:string – наименование банка из справочника по запросу getHVBanks в кодировке Unicode

branch:string (необязательное) – подразделение банка из справочника по запросу getHVBanks в кодировке Unicode, если есть

account:string – номер счета получателя

type:string (необязательное) – тип получателя user/settlement, по умолчанию «user»

address:string – USDT TRC 20 address для получателя settlement

source:string (необязательное) – по умолчанию «bank»

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку, отправлен
- 2 платеж отправлен, подтверждено принятие в обработку, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

Запрос статуса оплаты сервисов из предоплаченного баланса мерчанта (payout Вьетнам, Корея, Япония) – statusHVPayout

Конечная точка: **statusHVPayout**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startHVPayout

Запрос на оплату сервисов из предоплаченного баланса – startServicePay

Конечная точка: **startServicePay**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в копейках

currency: string – цифровой код валюты платежа по ISO 4217 (рубль – 643)

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

id_operator:integer – номер поставщика услуг из справочника, полученного запросом getServices

n:string – строка с информацией о платеже. Список необходимых параметров передается в результате запроса getServices. Например: 91233432534#12345#Проживание#Номер

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку,
- 2 платеж отправлен, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

Запрос статуса оплаты сервиса из предоплаченного баланса – statusServicePay

Конечная точка: **statusServicePay**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startServicePay

Запрос на оплату картой HUMO/UZCARD (payin Узбекистан) – startPWPayin

Конечная точка: **startPWPayin**

После окончания оплаты клиент автоматически перенаправляется на страницы url_success или url_fail в зависимости от успешности оплаты. Адреса страниц url_success и url_fail необходимо сообщить при подключении. В любой момент можно узнать текущий статус транзакции запросом statusPWPayin.

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в минимальные единицах платежа страны (Тийнах)*

currency: string – цифровой код валюты платежа по ISO 4217 (например UZS – 860)

id_trans: string – уникальный идентификатор транзакции на стороне клиента

n:string – строка с произвольной информацией о платеже. Обычно в ней параметры разделяются символом #. Первый параметр – идентификатор клиента, второй – код услуги по справочнику, остальные – по желанию. Например:

91233432534#12345#Проживание#Номер

В случае, если сервис поставщика успеет ответить в течение 30 секунд, то данные для проведения платежа клиентом будут сразу отображены. В противном случае необходимо повторять запросы statusPWPayin через несколько секунд (5-10).

В ответе будет url_redirect, на который необходимо перенаправить клиента для оплаты

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

url_redirect:string – ссылка на платежную страницу, на которую необходимо перенаправить клиента для оплаты картой

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку,
- 2 платеж отправлен, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент обработки запроса в ПС
- От 10 до 99 ошибочные окончательные

status_info – дополнительное текстовое описание статуса

Пример ответа:

{"id_trans":"1700401041192", "id_pay":"16662012", "status":"2", "url_redirect":"https://pay.1pay.uz/ru/checkout/T091QVRzV2taRTUzUGZsL0VWamx0QT09", "status_info":"Wait client pay", "rc":"0", "rm":"process"}

*** Важно:**

Лимиты на операцию «для примера»: Суточный 20 000 000; Месячный 50 000 000; Разовый 50 000-5 000 000. Лимиты указаны в Узбекских сумах. Необходимо передавать целое значение без "." или "," **умноженное на 100**.

Пример: надо передать 130 000 сум, Вы передаёте в поле "amount" значение "13000000", те: amount=13000000. **Таким образом передаваемое значение увеличивается на 00.** При этом если, сумму необходимо передать с копейками ее необходимо округлить до целых значений: надо передать 259 407,07 сум, Вы передаете в поле "amount", значение "25940800" сум, те: amount=25940800.

Все лимиты относятся только к успешным платежам, непосредственного плательщика, те физического лица.

При проведении тестов можно использовать следующие карты:

Номер карты (в таблице ниже), срок действия (любая дата больше текущей), CVV код – 456, СМС-ОТР – 12321.

Номер карты	Результат
8600860086008616	Успех
9860986098609811	
8600860086008624	Карта заблокирована
9860986098609829	
8600860086008632	SMS информирование не подключено
9860986098609837	
8600860086008640	Превышен лимит ввода PIN для карты
9860986098609845	

8600860086008657	Карта/Владелец в черном списке
9860986098609852	
8600860086008665	Недостаточно средств
9860986098609860	
8600860086008673	Операция проводится в ручном режиме. (технический сбой или задержка эквайера)
9860986098609878	
8600860086008691	Карта не найдена
9860986098609894	

Запрос статуса оплаты картой HUMO/UZCARD (payin Узбекистан) – statusPWPayin

Конечная точка: **statusPWPayin**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startPWPayin

Запрос на пополнение карт HUMO/UZCARD (payout Узбекистан) – startPWPayout

Конечная точка: **startPWPayout**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в минимальные единицах платежа страны (Тийнах)*

currency: string – цифровой код валюты платежа по ISO 4217 (узбекский сум – 860)

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

card:string – номер карты

name:string – имя плательщика

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку, отправлен
- 2 платеж отправлен, подтверждено принятие в обработку, ждем статус успешности.
- 3 проведен
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

Запрос статуса пополнения карт HUMO/UZCARD (payout Узбекистан) – statusPWPayout

Конечная точка: **statusPWPayout**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startPWPayout

Запрос на оплату картой Visa/MasterCard с PCI DSS – startVisaMastercardPayinPCIDSS

Конечная точка: **startVisaMastercardPayinPCIDSS**

ВНИМАНИЕ! Для передачи данных карты используется отдельный URL сервера, на котором не ведётся логгирование

Порядок оплаты:

1. Запрос инициализации платежа **startVisaMastercardPayinPCIDSS**
2. После получения status=1 в запросе инициализации или в ответе на запрос **statusVisaMastercardPayin** необходимо отправить данные карты запросом **sendCardInfo**
3. После успешной передачи данных карты в платёжную систему платеж переходит в статус 2 и находится в этом статусе до получения информации об успешном проведении (при этом переходит в статус 3) или информации об ошибке проведения (переход в статус 12 или 22 в зависимости от критичности)

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в минимальных единицах платежа страны (центах)

currency: string – цифровой код валюты платежа по ISO 4217 (USD – 840)

id_trans: string – уникальный идентификатор транзакции на стороне клиента

n:string – номер карты плательщика

В случае, если сервис поставщика услуги успеет ответить в течение 30 секунд, то платеж сразу перейдет в статус 1 и необходимо направить данные карты запросом sendCardInfo. В противном случае необходимо повторять запросы statusVisaMastercardPayin через несколько секунд (5-10).

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж принят на сервер.
- 0 определена платежная система.
- 1 запрос на платеж отправлен, необходимо передать данные карты запросом sendCardInfo
- 2 данные карты отправлены, ждем статус успешности.

- 3 проведен, платеж поступит. Фактическое поступление может быть до 48 часов в зависимости от банка получателя.
- Могут быть 100 101 102 в момент обработки запроса в ПС
- От 10 до 99 ошибочные окончательные

status_info – дополнительное текстовое описание статуса

Пример ответа:

{"id_trans":"1700401041192", "id_pay":"16662012", "status":"1", "status_info":"Wait card info", "rc":"0", "rm":"process"}

Передача данных карты при оплаты картой Visa/Mastercard с PCI DSS – sendCardInfo

Конечная точка: **sendCardInfo**

ВНИМАНИЕ! Для передачи данных карты используется отдельный URL сервера

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе

first_name – имя держателя карты

second_name – фамилия держателя карты

exp_month – месяц окончания срока действия карты

exp_year – две цифры года окончания действия карты

cvc – код карты

Параметры ответа:

Такие же как для startVisaMasterCardPayinPCIDSS

Запрос статуса оплаты картой Visa/Mastercard – statusVisaMastercardPayin

Конечная точка: **statusVisaMastercardPayin**

В момент получения информации о проведении платежа или ошибке направляется callback, поэтому запрос статуса делать не чаще раза в 10 минут на случай недоставки сообщения о статусе.

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startVisaMasterCardPayinPCIDSS

Запрос на пополнение карт Visa/Mastercard – startVisaMastercardPayout

Конечная точка: **startVisaMastercardPayout**

Дополнительные параметры запроса:

dt: DateTime – время платежа

amount: number – сумма в минимальных единицах платежа страны (центах)

currency: string – цифровой код валюты платежа по ISO 4217 (USD – 840)

id_trans:string(40) – уникальный идентификатор транзакции на стороне клиента

card:string – номер карты

first_name – имя держателя карты

second_name – фамилия держателя карты

exp_month – месяц окончания срока действия карты

exp_year – две цифры года окончания действия карты

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:string(40) – идентификатор транзакции клиента

id_pay:integer – идентификатор транзакции сервиса

status – этап проведения платежа

- -1 платеж только принят на сервер.
- 0 определена платежная система.
- 1 платеж прошел проверку, отправлен
- 2 платеж отправлен, подтверждено принятие в обработку, ждем статус успешности.
- 3 проведен, платеж поступит. Фактическое поступление может быть до 48 часов в зависимости от банка получателя.
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

Запрос статуса пополнения карт Visa/Mastercard – statusVisaMastercardPayout

Конечная точка: **statusVisaMastercardPayout**

Дополнительные параметры запроса:

id_pay:integer – идентификатор платежа на стороне сервиса, полученный в предыдущем запросе.

Параметры ответа:

Такие же как для startVisaMastercardPayout

Запрос доступного количества Giftcard для покупки – getServiceInfo

Конечная точка: **getServiceInfo**

Дополнительные параметры запроса:

id_service:integer – идентификатор сервиса id_service из запроса getServicesV2

или

id_group:integer – идентификатор группы сервиса id_group из запроса getServicesV2

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

items: service[] – список Giftcard в группе

service – {

id_service: integer – идентификатор сервиса

price: integer – стоимость Giftcard в центах

price_divider: integer – делитель стоимости

price_currency: integer – цифровой код валюты платежа по ISO 4217 (USD – 840)

price_kop: integer – сумма с учетом комиссии

inStock: integer – доступное количество Giftcard

}

Резерв требуемого количества Giftcard – reserveGiftcard

Конечная точка: **reserveGiftcard**

Дополнительные параметры запроса:

id_service:integer – идентификатор сервиса id_service из запроса getServicesV2

n:string – описание платежа (например идентификатор заказа клиента)

id_trans:string – уникальный идентификатор запроса на резервирование

дополнительно передаются параметры для резервирования, полученные в результатах запроса getServicesV2, например:

quantity:integer – количество покупаемых Giftcards

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_reserve:integer – идентификатор резерва Giftcards

price:integer – сумма к оплате в центах

price_divider:integer – делитель суммы к оплате

price_currency - цифровой код валюты платежа по ISO 4217 (USD – 840)

Подтверждение оплаты резерва Giftcard – confirmGiftcardPaid

Конечная точка: **confirmGiftcardPaid**

Дополнительные параметры запроса:

id_reserve:integer – идентификатор резерва из результата запроса reserveGiftcard

amount_paid:integer – оплаченная сумма в центах

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_pay:integer – идентификатор giftCard

status:integer –этап получения GiftCard

- -1 запрос принят в обработку
- 0 в процессе запроса Giftcard
- 2 запрос отправлен, GiftCard не получены
- 3 Giftcard получены, можно их получать запросом statusGiftcard
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

Запрос статуса Giftcard, а также их реквизитов – statusGiftcard

Конечная точка: **statusGiftcard**

Дополнительные параметры запроса:

id_pay:integer – идентификатор giftCard

Параметры ответа:

rc: number – код ошибки (0 – успешно)

rm: string – текстовое описание ошибки (в случае если rc не 0)

id_trans:integer – идентификатор запроса, переданный в reserveGiftcard

id_pay:integer – идентификатор giftCard

id_reserve:integer – идентификатор резерва

status:integer –этап получения GiftCard

- -1 запрос принят в обработку
- 0 в процессе запроса Giftcard
- 2 запрос отправлен, GiftCard не получены
- 3 Giftcard получены, можно их получать запросом statusGiftcard
- Могут быть 100 101 102 в момент запроса в ПС
- От 10 до 99 ошибочные окончательные

status_info:string – описание статуса

giftcards: giftcard[] – список реквизитов Giftcard

giftcard – {

description:string – инструкция по использованию Giftcard

serialNumber:string – номер Giftcard

pin:string – PIN для Giftcard

expiration:string – срок действия Giftcard

}